

中南大學
CENTRAL SOUTH UNIVERSITY

NLP 实验报告

课　　程：	自然语言处理
学生姓名：	宣丽娜
指导教师：	李芳芳
学　　院：	计算机学院
专业班级：	计科 T2401 班

2026 年 6 月

目 录

一、实验目的 4

二、实验环境 5

三、实验总体原理 6

3.1 中文文本处理整体流程 6

3.2 中文分词与文本预处理 6

3.3 文本向量化表示 6

四、实验一：中文文本分词与预处理 7

4.1 实验目的 7

4.2 实验原理 7

4.3 实验内容与代码实现 7

4.3.1 三种分词模式与词性标注 7

4.3.2 人物别名归一化 8

4.3.3 高频词统计与人物统计 9

4.3.4 自定义词典分词效果对比 10

4.4 实验结果与分析 11

五、实验二：文本向量化表示 12

5.1 实验目的 12

5.2 实验原理 12

5.3 实验内容与代码实现 12

5.3.1 读取停用词与中文文本预处理 12

5.3.2 词袋模型向量化 13

5.3.3 TF-IDF 权重转换 13

5.3.4 一步式 TF-IDF 向量化与新闻文本向量化 13

5.3.5 主程序测试 14

5.4 实验结果与分析 14

六、综合分析 16

6.1 两个实验之间的关系 16

6.2 分词预处理对向量化结果的影响 16

6.3 词袋模型与 TF-IDF 的比较 16

七、实验总结 17

八、参考资料 18

九、附源代码..... 19

一、实验目的

理解中文自然语言处理的基本流程，掌握从原始中文文本到结构化特征表示的完整处理思路。

掌握 Jieba 分词工具的安装与基本使用方法，理解精确模式、全模式和搜索引擎模式的区别。

掌握中文文本预处理方法，包括停用词过滤、去除单字词、词性标注、人物别名归一化和自定义词典使用。

掌握高频词统计和主要人物出场次数统计方法，能够从文本中提取具有解释意义的信息。

掌握词袋模型、TF-IDF 加权方法以及 CountVectorizer、TfidfTransformer、TfidfVectorizer 等工具的使用。

能够将中文文本经过分词和预处理后转换为数值特征矩阵，为后续文本分类、聚类、情感分析和信息检索任务奠定基础。

二、实验环境

项目	内容
操作系统	Windows 或 Linux
开发工具	PyCharm
编程语言	Python
主要依赖库	jieba、jieba.posseg、numpy、scikit-learn
实验数据	《三国演义》文本 threekingdoms.txt；新闻文本 news.txt；停用词表 stop_words.txt

三、实验总体原理

3.1 中文文本处理整体流程

本综合实验围绕中文自然语言处理的基础流程展开，整体上包括中文文本分词与预处理、文本统计分析以及文本向量化表示三个阶段。中文文本通常没有天然空格分隔，因此需要先利用分词工具将连续文本切分为词语序列；随后通过停用词过滤、别名归一化和自定义词典等方法提升文本表示质量；最后将文本转换为计算机可处理的数值矩阵，用于后续机器学习或信息检索任务。

整体流程可以概括为：原始文本 → 中文分词 → 停用词过滤 → 词性标注/别名归一化 → 词频统计 → 词袋模型/TF-IDF 向量化 → 后续分类、聚类或检索任务。

3.2 中文分词与文本预处理

中文文本中词语之间通常没有显式分隔符，因此在进行文本分析前必须先完成中文分词。Jieba 分词工具能够基于词典和统计模型将连续中文文本切分为词语序列，并提供精确模式、全模式和搜索引擎模式三种分词方式。

文本预处理是提高后续统计和向量化质量的重要环节。本实验主要采用去除单字词、停用词过滤、人物别名归一化和自定义词典等方法。其中，人物别名归一化可以避免同一人物被统计为多个不同词条；自定义词典可以提高专业术语和专有名词的识别准确性。

3.3 文本向量化表示

文本向量化是将自然语言文本转换为数值特征矩阵的过程，是文本分类、聚类、情感分析等任务的基础。词袋模型通过统计词语出现次数构建词频矩阵，方法简单直观，但忽略词序和上下文语义。TF-IDF 在词频基础上引入逆文档频率，能够降低常见词影响，突出更具区分能力的关键词。

四、实验一：中文文本分词与预处理

4.1 实验目的

理解中文文本分词的基本概念和流程。

掌握 Jieba 分词工具的基本使用方法。

掌握 Jieba 的精确模式、全模式和搜索引擎模式。

掌握词性标注、停用词过滤、词频统计和人物别名归一化。

能够对《三国演义》文本进行高频词和主要人物统计分析。

4.2 实验原理

Jieba 精确模式试图将句子最准确地切开，适合常规文本分析；全模式会扫描句子中所有可能成词的词语，速度较快但存在冗余；搜索引擎模式在精确模式基础上对长词再次切分，适合搜索引擎建立索引。

在《三国演义》人物统计中，文本中的“孟德”“丞相”可归一化为“曹操”，“孔明”“孔明曰”可归一化为“诸葛亮”，“玄德”“玄德曰”可归一化为“刘备”，“关公”“云长”可归一化为“关羽”。通过该处理可以减少同一实体的重复统计问题。

4.3 实验内容与代码实现

4.3.1 三种分词模式与词性标注

精确模式分词

```
import jieba

def precise_seg(s):
    return jieba.lcut(s, cut_all=False)

s = '中南大学信息内容安全课程实验'
print('精确模式：', precise_seg(s))
```

结果说明：精确模式适合常规文本分析，能够较准确地输出词语序列。

全模式分词

```
def full_seg(s):
    return jieba.lcut(s, cut_all=True)

print('全模式：', full_seg(s))
```

结果说明：全模式会输出句子中可能存在的多个词语，结果较全面但冗余较多。

搜索引擎模式分词

```
def search_seg(s):
    return jieba.lcut_for_search(s)

print('搜索模式：', search_seg(s))
```

结果说明：搜索引擎模式适合检索场景，有助于提高搜索召回率。

词性标注

```
import jieba.posseg as pseg

def pos_seg(s):
    return pseg.lcut(s)

print('词性标注：', pos_seg(s))
```

结果说明：词性标注可以输出词语及其词性，为后续实体识别和句法分析提供基础。

运行结果：

```
精确模式： ['中南大学', '信息内容', '安全', '课程', '实验']
全模式： ['中南', '中南大学', '南大', '大学', '信息', '信息内容', '内容', '安全', '课程', '实验']
搜索模式： ['中南', '南大', '大学', '中南大学', '信息', '内容', '信息内容', '安全', '课程', '实验']
词性标注： [pair('中南大学', 'nt'), pair('信息内容', 'n'), pair('安全', 'an'), pair('课程', 'n'), pair('实验', 'vn')
]
```

结果说明：精确模式适合常规文本分析；全模式结果较全面但冗余较多；搜索引擎模式有助于提高检索召回率；词性标注可以为后续实体识别和句法分析提供基础。

4.3.2 人物别名归一化

```
def normalize_name(w):
    if w in ['孟德', '丞相']:
        return '曹操'
    elif w in ['玄德', '玄德曰']:
        return '刘备'
    elif w in ['孔明', '孔明曰']:
        return '诸葛亮'
    elif w in ['关公', '云长']:
        return '关羽'
    return w
```

结果说明：通过人物别名归一化，可以避免同一人物被统计为多个不同词条。

4.3.3 高频词统计与人物统计

```
def top20_words():
    txt = open(r'D:\作业\nlp\data\threekingdoms.txt', 'r', encoding='utf-8').read()
    words = jieba.lcut(txt)
    counts = {}
    stopwords = {'将军', '却说', '荆州', '二人', '不可', '不能', '如此', '今日',
                 '次日', '于是', '如何', '主公', '一人', '不敢', '大喜', '陛下',
                 '不知', '汉中', '只见', '都督', '后主'}
    for w in words:
        if len(w) == 1:
            continue
        if w in stopwords:
            continue
        w = normalize_name(w)
        counts[w] = counts.get(w, 0) + 1
    li = list(counts.items())
    li.sort(key=lambda x: x[1], reverse=True)
    print('三国演义前 20 高频词：')
    for i in range(20):
        print('{:<3}{:<8}{:>6}'.format(i + 1, li[i][0], li[i][1]))
```

结果说明：该程序先分词，再过滤单字词和停用词，最后按词频降序输出前 20 个高频词。

运行结果：

```
三国演义前20高频词：
1  曹操      1418
2  诸葛亮     1363
3  刘备      1213
4  关羽       779
5  张飞       348
6  商议       340
7  军士       306
8  吕布       299
9  左右       289
10 军马       288
11 引兵       275
12 孙权       263
13 赵云       254
14 天下       250
15 东吴       249
16 魏兵       233
17 司马懿      221
18 人马       218
19 周瑜       217
20 众将       205
```

```
def people_count():
    txt = open(r'D:\作业\nlp\data\threekingdoms.txt', 'r', encoding='utf-8').read()
    words = jieba.lcut(txt)
    counts = {}
    people = ['曹操', '诸葛亮', '刘备', '关羽', '张飞', '吕布', '赵云', '姜维',
              '周瑜', '孙权', '司马懿', '黄忠', '马超', '魏延', '庞统', '袁绍']
    for w in words:
        if len(w) == 1:
            continue
        w = normalize_name(w)
        counts[w] = counts.get(w, 0) + 1
    result = {k: v for k, v in counts.items() if k in people}
    li = list(result.items())
    li.sort(key=lambda x: x[1], reverse=True)
    print('主要人物出场次数：')
    for name, count in li:
        print('{:<8}{:>6}'.format(name, count))
```

结果说明：人物出场次数可以反映小说文本中的人物重要程度和叙事重心。

运行结果：

```
主要人物出场次数：
曹操      1418
诸葛亮      1363
刘备      1213
关羽       779
张飞       348
吕布       299
孙权       263
赵云       254
司马懿       221
周瑜       217
袁绍       189
马超       183
魏延       177
黄忠       168
姜维       151
庞统        69
```

4.3.4 自定义词典分词效果对比

```
import jieba

s = "中南大学信息内容安全课程实验"

print("未加入自定义词典：", jieba.lcut(s))

jieba.add_word("信息内容安全")
```

```
jieba.add_word("中南大学")

print("加入自定义词典后：", jieba.lcut(s))
```

结果分析：

未加入自定义词典时，jieba 可能会将专业术语拆分成多个词；加入自定义词典后，“中南大学”“信息内容安全”等专有词能够作为整体被识别，分词结果更符合实验语境，说明自定义词典可以提高专业领域文本分词的准确性。

运行结果：

```
(.venv) PS D:\作业> & d:/作业/.venv/Scripts/python.exe d:/作业/nlp/data/test.py
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\59718\AppData\Local\Temp\jieba.cache
Loading model cost 0.549 seconds.
Prefix dict has been built successfully.
未加入自定义词典：  ['中南大学', '信息内容', '安全', '课程', '实验']
加入自定义词典后：  ['中南大学', '信息内容安全', '课程', '实验']
```

4.4 实验结果与分析

实验内容	输出结果	分析
精确模式分词	词语切分结果	切分较准确，适合文本分析。
全模式分词	所有可能成词结果	结果较多，但冗余明显。
搜索模式分词	适合搜索的切分结果	有利于建立搜索索引，提高召回率。
词性标注	词语及对应词性	可用于后续实体识别和句法分析。
高频词统计	前 20 个高频词	能够体现文本主题和主要叙事对象。
人物统计	主要人物出场次数	能够反映人物在文本中的重要程度。
自定义词典	加入词典前后分词对比	能够提高专有词和专业术语识别准确性。

综合实验结果可知，文本分词和预处理对后续文本分析具有基础性作用。若分词粒度不合理或专有词被错误拆分，将直接影响词频统计和实体统计结果。因此，在中文文本分析中，需要结合文本领域特点适当加入自定义词典，并通过停用词过滤和别名归一化提高统计结果的准确性。

五、实验二：文本向量化表示

5.1 实验目的

理解文本向量化的基本概念和意义。

掌握词袋模型的原理及 CountVectorizer 的使用方法。

掌握 TF-IDF 加权方法的原理及 TfidfTransformer 的使用方法。

掌握 TfidfVectorizer 一步式向量化流程。

能够将中文文本经过分词和预处理后转换为向量矩阵。

5.2 实验原理

文本向量化是将自然语言文本转换为计算机可处理的数值向量的过程，是后续文本分类、聚类和情感分析等任务的基础。

词袋模型忽略词语顺序，只统计每个词语在文档中的出现次数。CountVectorizer 可以将语料库转换为词频矩阵，矩阵的行表示文档，列表示特征词。

TF-IDF 由词频 TF 和逆文档频率 IDF 构成。词语在某篇文档中出现越多，TF 越高；词语在整个语料库中出现的文档越少，IDF 越高。因此，TF-IDF 能够突出具有区分能力的关键词。

5.3 实验内容与代码实现

5.3.1 读取停用词与中文文本预处理

读取停用词

```
def load_stopwords(path):
    with open(path, 'r', encoding='utf-8') as f:
        return [line.strip() for line in f]
```

结果说明：读取停用词表，为后续文本过滤提供基础。

中文文本预处理

```
import jieba

def preprocess_text(text, stopwords):
    words = jieba.lcut(text.strip())
    return ' '.join([w for w in words if w not in stopwords and len(w) > 1])
```

结果说明：该函数先对文本进行分词，再去除停用词和单字词，最后输出空格分隔的词语序列

5.3.2 词袋模型向量化

```
from sklearn.feature_extraction.text import CountVectorizer

def bow_vectorize(corpus):
    vectorizer = CountVectorizer()
    X = vectorizer.fit_transform(corpus)
    print('词袋模型特征词：', vectorizer.get_feature_names_out())
    print('词频矩阵：')
    print(X.toarray())
    return X, vectorizer
```

结果说明：CountVectorizer 将分词后的文本转换为词频矩阵，可以直观表示词语出现次数。

5.3.3 TF-IDF 权重转换

```
import numpy as np
from sklearn.feature_extraction.text import TfidfTransformer

def tfidf_transform(X_count):
    transformer = TfidfTransformer()
    X_tfidf = transformer.fit_transform(X_count)
    print('TF-IDF 权重矩阵：')
    print(np.round(X_tfidf.toarray(), 4))
    return X_tfidf
```

结果说明：TfidfTransformer 在词频矩阵基础上计算 TF-IDF 权重，使关键词具有更高权重。

5.3.4 一步式 TF-IDF 向量化与新闻文本向量化

```
from sklearn.feature_extraction.text import TfidfVectorizer

def tfidf_vectorize(corpus):
    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform(corpus)
    print('特征维度：', X.shape)
    print('特征词：', vectorizer.get_feature_names_out()[:20])
    return X, vectorizer
```

结果说明：TfidfVectorizer 可以直接从文本得到 TF-IDF 矩阵，简化了文本向量化流程。

```
def news_vectorization(corpus_path, stopwords_path):
    stopwords = load_stopwords(stopwords_path)
    corpus = []
    with open(corpus_path, 'r', encoding='utf-8') as f:
        for line in f:
            corpus.append(preprocess_text(line, stopwords))
    vectorizer = TfidfVectorizer(max_features=5000)
    X = vectorizer.fit_transform(corpus)
    print('文档数：', X.shape[0])
    print('特征数：', X.shape[1])
    print('前 10 个特征词：', vectorizer.get_feature_names_out()[:10])
    return X, vectorizer
```

结果说明：该函数适用于新闻语料，将多个新闻文本转换为 TF-IDF 特征矩阵。

5.3.5 主程序测试

```
if __name__ == '__main__':
    sample_corpus = [
        '这是一个测试文档',
        '这是第二个测试文档',
        '文档向量化是文本处理的重要步骤'
    ]
    X_count, vec = bow_vectorize(sample_corpus)
    tfidf_transform(X_count)
    tfidf_vectorize(sample_corpus)
    # news_vectorization('data/news.txt', 'data/stop_words.txt')
```

结果说明：主程序依次测试词袋模型、TF-IDF 转换和一步式 TF-IDF 向量化。

运行结果：

```
(.venv) PS D:\作业> & d:/作业/.venv/Scripts/python.exe d:/作业/nlp/ba.py
词袋模型特征词： ['文档向量化是文本处理的重要步骤' '这是一个测试文档' '这是第二个测试文档']
词频矩阵：
[[0 1 0]
 [0 0 1]
 [1 0 0]]
TF-IDF 权重矩阵：
[[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]]
特征维度： (3, 3)
特征词： ['文档向量化是文本处理的重要步骤' '这是一个测试文档' '这是第二个测试文档']
```

5.4 实验结果与分析

实验内容	主要输出	分析
文本预处理	分词后且去除停用词的语料	减少无意义词语对后续统计和建模的影响。
词袋模型	特征词列表和词频矩阵	实现简单，但忽略词序和语义关系。
TF-IDF 转换	TF-IDF 权重矩阵	能够突出区分度较高的关键词。
一步式 TF-IDF	特征维度和部分特征词	流程简洁，适合实际工程应用。

新闻文本向量化	文档数、特征数、前 10 个特征词	可为文本分类、聚类等任务提供数值特征。

实验结果表明，词袋模型能够直观表示词语出现次数，但无法体现词语的重要程度。TF-IDF 在词频基础上引入逆文档频率，可以降低高频常见词的影响，使具有区分能力的关键词获得更高权重。相比词袋模型，TF-IDF 更适合文本分类、聚类和信息检索等实际任务。

六、综合分析

6.1 两个实验之间的关系

实验一和实验二并不是相互独立的内容，而是中文自然语言处理流程中的前后两个阶段。实验一完成从原始中文文本到词语序列的转换，并通过停用词过滤、别名归一化和自定义词典优化文本质量；实验二则在此基础上进一步将文本转换为数值特征矩阵，使文本能够被机器学习算法处理。

因此，实验一解决的是“文本如何被切分和清洗”的问题，实验二解决的是“文本如何被计算机表示和建模”的问题。前者是后者的基础，后者是前者的进一步应用。

6.2 分词预处理对向量化结果的影响

中文文本向量化高度依赖分词结果。如果分词阶段将一个专有名词错误拆分，向量化阶段就会把它作为多个不同特征处理，从而影响词频矩阵和 TF-IDF 权重。例如，“信息内容安全”如果被拆分为“信息”“内容”“安全”，则无法准确表达完整专业术语的含义；加入自定义词典后，该词能够作为一个整体进入特征空间，提高文本表示的准确性。

停用词过滤也会影响向量化质量。过多无意义词进入特征矩阵，会增加特征维度并稀释关键词权重；而合理过滤停用词可以减少噪声，使 TF-IDF 更准确地突出文本主题。

6.3 词袋模型与 TF-IDF 的比较

比较项目	词袋模型	TF-IDF
基本思想	统计词语出现次数	综合词频和逆文档频率计算权重
优点	简单直观，容易理解和实现	能够突出区分度较高的关键词
缺点	无法体现词语重要程度，忽略上下文语义	仍不能直接表达深层语义和词序关系
适用场景	基础词频统计、简单文本表示	文本分类、聚类、检索和关键词提取
与预处理关系	依赖分词和停用词过滤质量	对停用词、低频词和特征选择更加敏感

七、实验总结

本综合实验按照“先融合、再分开、最后融合”的思路，将中文文本分词与预处理实验、文本向量化表示实验整合为一个完整的中文自然语言处理基础实验。

通过实验一，掌握了 Jieba 分词工具的三种分词模式、词性标注、停用词过滤、人物别名归一化、自定义词典和词频统计等方法，理解了中文文本预处理在文本分析中的基础作用。通过实验二，掌握了 CountVectorizer、TfidfTransformer 和 TfidfVectorizer 的使用方法，理解了词袋模型与 TF-IDF 在文本数值化表示中的作用。

综合来看，中文文本分析的关键流程可以概括为：原始文本首先经过分词和预处理形成较规范的词语序列，再通过词袋模型或 TF-IDF 转换为数值特征矩阵，最终服务于分类、聚类、情感分析和信息检索等任务。实验结果说明，合理的文本预处理能够显著影响后续向量化质量，而合适的向量化方法则决定了文本特征能否有效表达文本内容。

八、参考资料

[1]实验指导书：实验 1 中文文本分词与预处理。
[2]实验指导书：实验 2 文本向量化表示。
[3]Jieba 中文分词工具相关文档。
[4]Scikit-learn 文本特征提取相关文档。

九、附源代码

以下代码将两个实验内容合并到同一个 Python 文件中，先完成中文分词、预处理与统计分析，再完成词袋模型和 TF-IDF 向量化。

附源代码：

```
import jieba
import jieba.posseg as pseg

def precise_seg(s):
    return jieba.lcut(s, cut_all=False)

def full_seg(s):
    return jieba.lcut(s, cut_all=True)

def search_seg(s):
    return jieba.lcut_for_search(s)

def pos_seg(s):
    return pseg.lcut(s)

def normalize_name(w):
    if w in ['孟德', '丞相']:
        return '曹操'
    elif w in ['玄德', '玄德曰']:
        return '刘备'
    elif w in ['孔明', '孔明曰']:
        return '诸葛亮'
    elif w in ['关公', '云长']:
        return '关羽'
    return w

def top20_words():
    txt = open(r'D:\作业\nlp\data\threekingdoms.txt', 'r', encoding='utf-8').read()
    words = jieba.lcut(txt)

    counts = {}

    stopwords = {
        '将军', '却说', '荆州', '二人', '不可', '不能', '如此', '今日',
        '次日', '于是', '如何', '主公', '一人', '不敢', '大喜', '陛下',
        '不知', '汉中', '只见', '都督', '后主'
```

```
    }

    for w in words:
        if len(w) == 1:
            continue

        if w in stopwords:
            continue

        w = normalize_name(w)
        counts[w] = counts.get(w, 0) + 1

    li = list(counts.items())
    li.sort(key=lambda x: x[1], reverse=True)

    print('三国演义前 20 高频词：')
    for i in range(20):
        print('{:<3}{:<8}{:>6}'.format(i + 1, li[i][0], li[i][1]))

def people_count():
    txt = open(r'D:\作业\nlp\data\threekingdoms.txt', 'r', encoding='utf-8').read()
    words = jieba.lcut(txt)

    counts = {}

    people = [
        '曹操', '诸葛亮', '刘备', '关羽', '张飞', '吕布', '赵云', '姜维',
        '周瑜', '孙权', '司马懿', '黄忠', '马超', '魏延', '庞统', '袁绍'
    ]

    for w in words:
        if len(w) == 1:
            continue

        w = normalize_name(w)
        counts[w] = counts.get(w, 0) + 1

    result = {k: v for k, v in counts.items() if k in people}
    li = list(result.items())
    li.sort(key=lambda x: x[1], reverse=True)

    print('主要人物出场次数：')
    for name, count in li:
```

```
        print('{:<8}{:>6}'.format(name, count))

if __name__ == '__main__':
    s = '中南大学信息内容安全课程实验'

    print('精确模式：', precise_seg(s))
    print('全模式：', full_seg(s))
    print('搜索模式：', search_seg(s))
    print('词性标注：', pos_seg(s))

    top20_words()
    people_count()
```

```
import jieba
import numpy as np

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.feature_extraction.text import TfidfVectorizer


def load_stopwords(path):
    with open(path, 'r', encoding='utf-8') as f:
        return [line.strip() for line in f]


def preprocess_text(text, stopwords):
    words = jieba.lcut(text.strip())
    return ' '.join([w for w in words if w not in stopwords and len(w) > 1])


def bow_vectorize(corpus):
    # 词袋模型：CountVectorizer
    vectorizer = CountVectorizer()
    X = vectorizer.fit_transform(corpus)

    print('词袋模型特征词：', vectorizer.get_feature_names_out())
    print('词频矩阵：')
    print(X.toarray())

    return X, vectorizer
```

```python
def tfidf_transform(X_count):
    # TF-IDF 转换
    transformer = TfidfTransformer()
    X_tfidf = transformer.fit_transform(X_count)

    print('TF-IDF 权重矩阵：')
    print(np.round(X_tfidf.toarray(), 4))

    return X_tfidf


def tfidf_vectorize(corpus):
    # 一步完成 TF-IDF 向量化
    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform(corpus)

    print('特征维度：', X.shape)
    print('特征词：', vectorizer.get_feature_names_out()[:20])

    return X, vectorizer


def news_vectorization(corpus_path, stopwords_path):
    stopwords = load_stopwords(stopwords_path)
    corpus = []

    with open(corpus_path, 'r', encoding='utf-8') as f:
        for line in f:
            corpus.append(preprocess_text(line, stopwords))

    # TF-IDF 向量化
    vectorizer = TfidfVectorizer(max_features=5000)
    X = vectorizer.fit_transform(corpus)

    print('文档数：', X.shape[0])
    print('特征数：', X.shape[1])
    print('前 10 个特征词：', vectorizer.get_feature_names_out()[:10])

    return X, vectorizer


if __name__ == '__main__':
    sample_corpus = [
        '这是一个测试文档',
        '这是第二个测试文档',
        '文档向量化是文本处理的重要步骤'
```

```
]

# 示例 1：词袋模型
X_count, vec = bow_vectorize(sample_corpus)

# 示例 2：TF-IDF 转换
tfidf_transform(X_count)

# 示例 3：一步 TF-IDF
tfidf_vectorize(sample_corpus)

# 示例 4：新闻文本向量化，需准备数据文件
# news_vectorization('data/news.txt', 'data/stop_words.txt')ppip
```